Software Development: Hospital Project

Overview & Design

17 Mar 2026

Overview **2**
Functional Requirements **2**
1. Patient & Insurance Management 2
2. Clinical Operations (Doctor Tasks) 2
3. Data Retrieval & Reporting 3
4. System Administration 3
Non-Functional Requirements **3**
Task & User Analysis **4**
Key Task Flows **4**
Designs **5**
Class Diagram 5
Use Case Diagram 6
Activity Diagram 7
Recording Visits 7
Issuing a new prescription 8
Phase 1: Some Thoughts **8**

Overview

The primary goal is to create a robust, object-oriented software solution that manages the lifecycle of medical data. The system is designed to handle different types of users. Such as **Doctors**, **Admins**, and **Insurance Admins.** to ensure that data is recorded and retrieved accurately while maintaining a high standard of programming practice.

- **Platform**: Java (Object-Oriented Paradigm).
- **Database**: MariaDB (Hospital database).
- **Core Logic**: Implementation of inheritance, encapsulation (private variables), and polymorphism.
- **Key Deliverables**: UML diagrams (Class and Use Case), source code, a testing plan, a user guide, and an evaluation report.

Functional Requirements

The functional requirements define what the system must do to support the business needs of the health insurance company.

1. Patient & Insurance Management

a. **Register Patients**: The system must store patient names, addresses, phone numbers, and emails.
b. **Track Insurance**: Each patient must be linked to a primary care doctor and an insurance company via an insurance ID number.
c. **Verify Eligibility**: The system must allow an Insurance Admin to verify patient coverage and manage provider lists.

2. Clinical Operations (Doctor Tasks)

a. **Record Visits**: Doctors must be able to log patient visits, including the date of the visit, symptoms reported, and the diagnosis given.
b. **Issue Prescriptions**: The system must track prescribed drugs, including the drug name, dosage, duration, and purpose.
c. **Safety Checks**: The application should provide information on a drug's possible side effects to ensure there are no conflicts with a patient's other medications.

3. Data Retrieval & Reporting

a. **Medical History**: Users must be able to retrieve historical visit and prescription data.
b. **Search Filters**: The system should allow for filtered searches, such as viewing all prescriptions for a specific patientID or viewing the most recent entries.
c. **Trend Tracking**: The accumulated data will eventually be used to track health trends and perform modeling.

4. System Administration

a. **Staff Management**: Admins must be able to manage doctor profiles, including their specialties and contact information.
b. **Database Maintenance**: Admins are responsible for updating the drug database and ensuring system integrity.

Non-Functional Requirements

These define the operational constraints and quality attributes of the system. Not all will be part of the system and are things that will be implemented in the future.

- **Security**: All sensitive medical data must be encrypted at rest and in transit. Access is strictly role-based, ensuring Doctors cannot perform Admin tasks and vice versa.
- **Data Integrity**: The MariaDB backend must use ACID-compliant transactions to ensure that patient records are never partially saved or corrupted during a crash.
- **Performance**: The Dynamic Query Builder must return search results in under 2 seconds to maintain efficiency in a fast-paced medical environment.
- **Usability**: The interface must be intuitive enough for medical staff to record a visit with minimal typing.
- **Scalability**: The system architecture should allow for an increasing number of patient records and concurrent users without a significant dip in response time.

Task & User Analysis

This section identifies who is using the system and the core tasks they must accomplish.

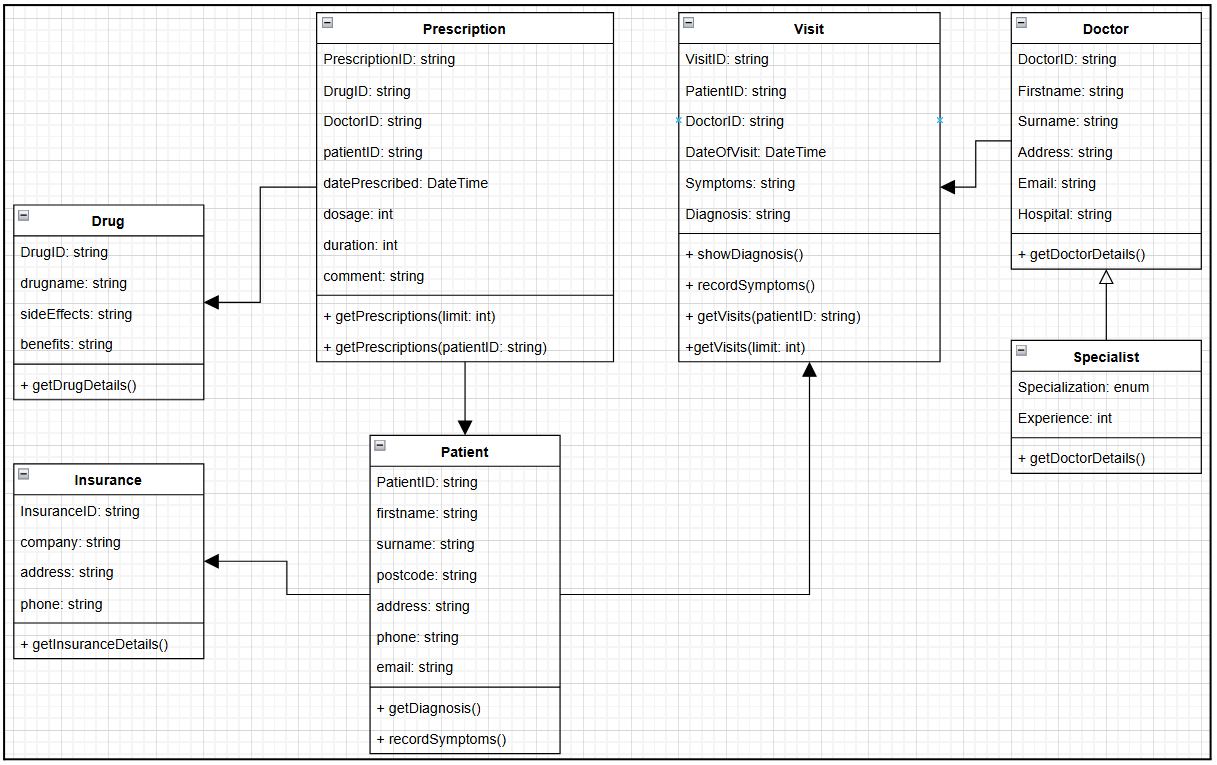
Persona	Goal	Technical Skill
Doctor	Quickly log patient symptoms and safely prescribe medication.	Moderate; requires a streamlined workflow.
Insurance Admin	Verify patient eligibility and manage insurance provider details.	High; focused on data accuracy and compliance.
System Admin	Maintain the drug database and manage staff access levels.	Expert; requires full system visibility.

Key Task Flows

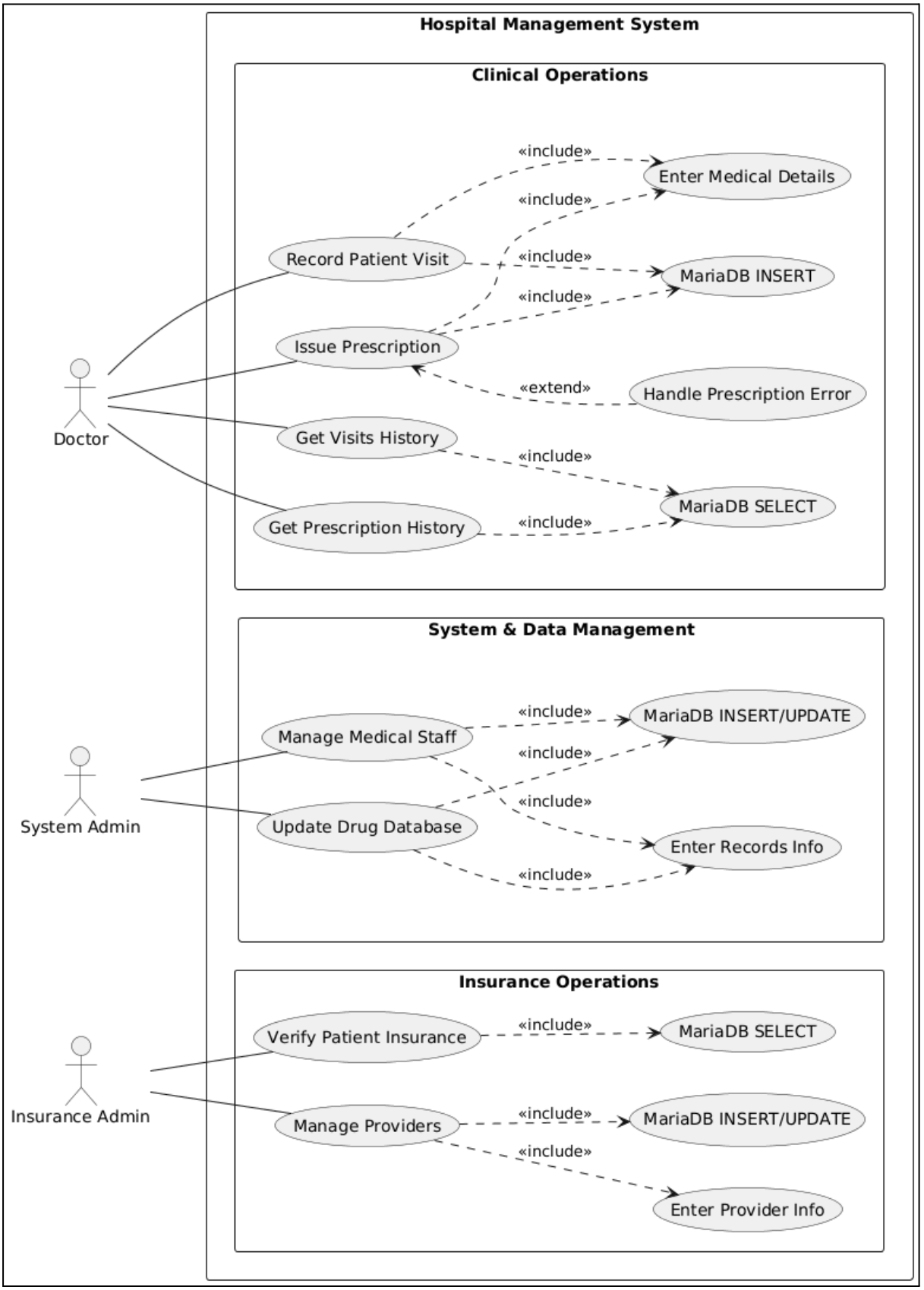
1. Patient Verification (Doctor/Admin):
    - Goal: Access a patient record.
    - Action: Input "Rule of Three" identifiers (First Name, Surname, Address) to unlock the medical history.
2. Clinical Documentation (Doctor):
    - Goal: Record a visit.
    - Action: Select a patient, input symptoms, and use the safety check feature to ensure a new prescription doesn't conflict with existing meds.
3. Database Maintenance (System Admin):
    - Goal: Update the drug repository.
    - Action: Upload new drug profiles, including dosage guidelines and known side effects.

Designs

Class Diagram

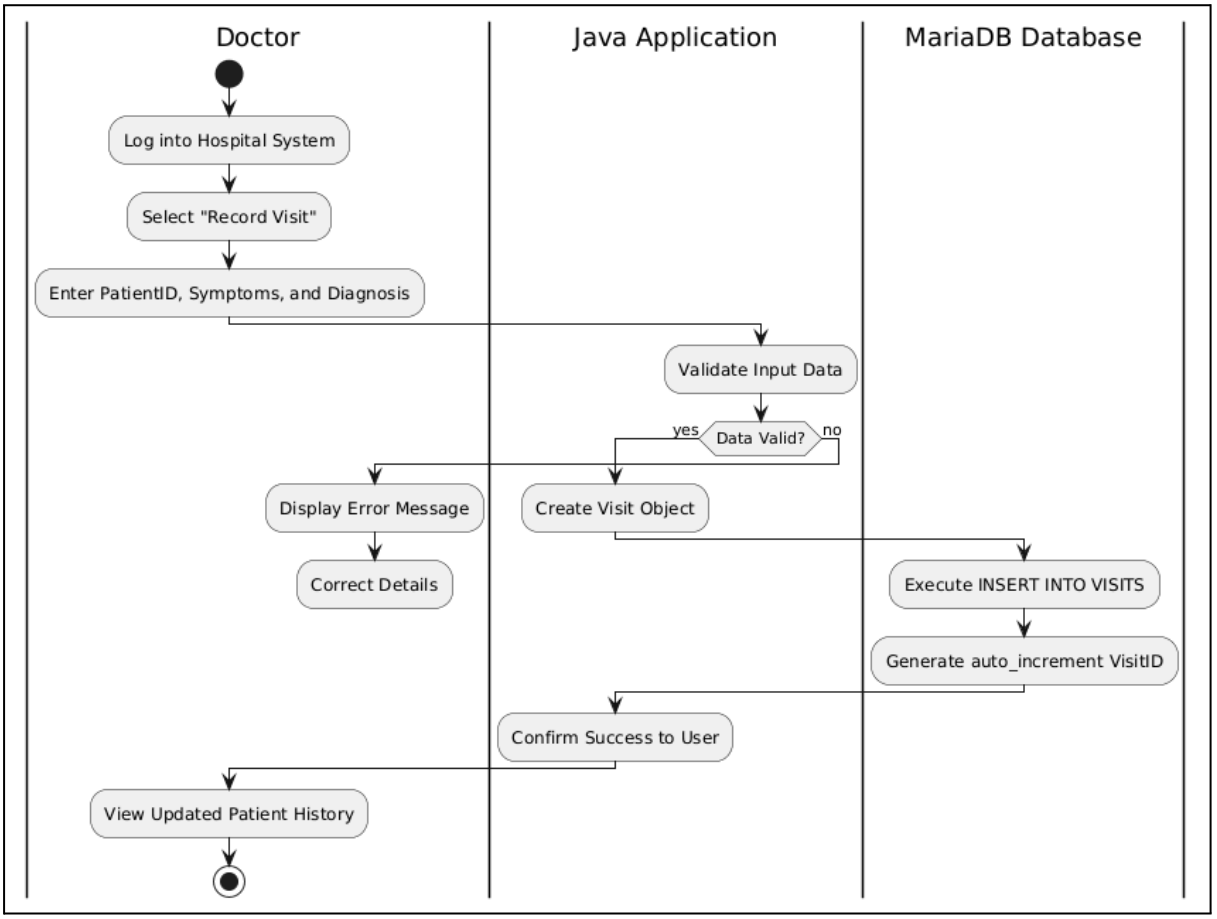


Use Case Diagram

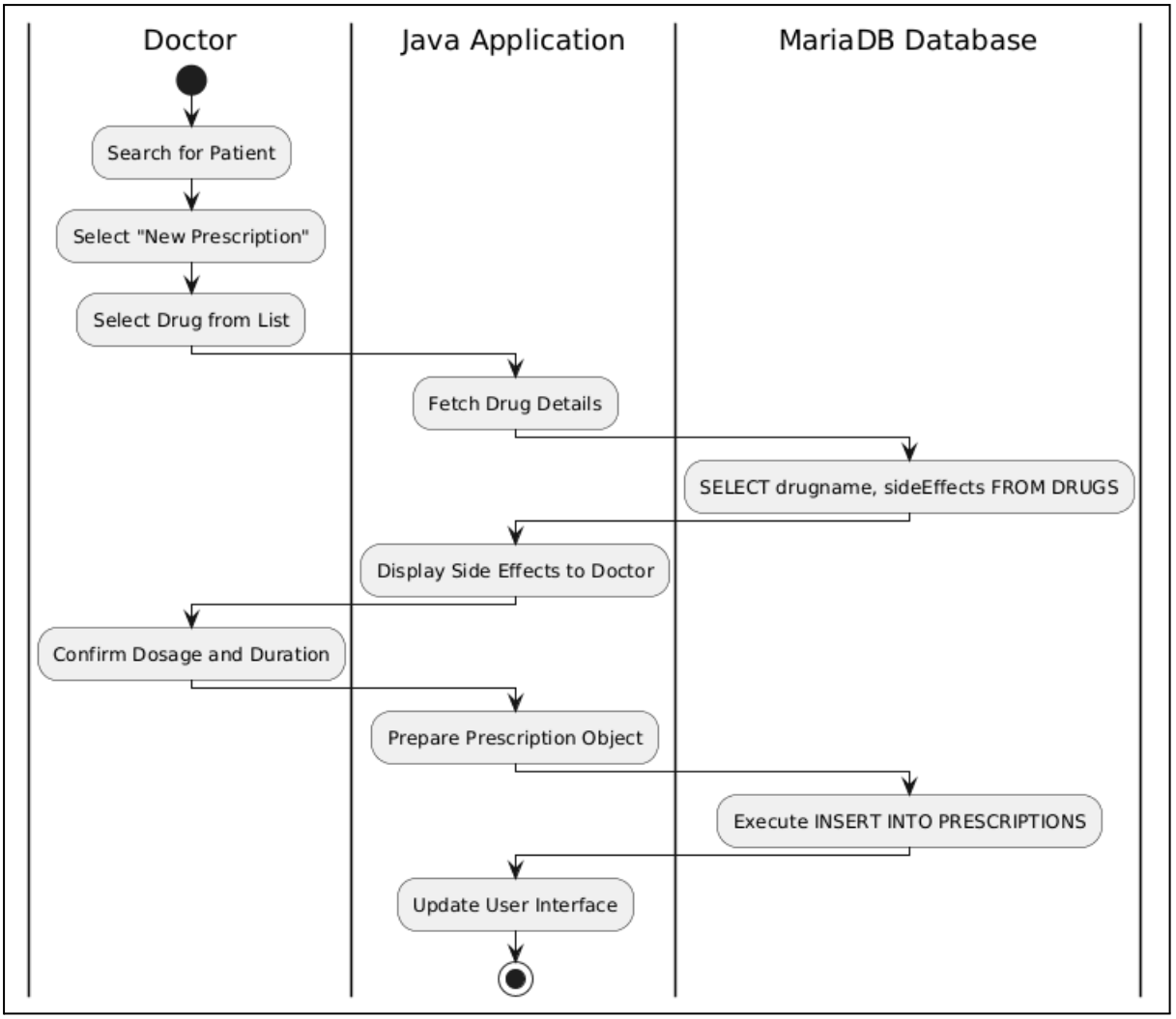


Activity Diagram

Recording Visits



Issuing a new prescription



Phase 1: Some Thoughts

I'm thinking of integrating a **Dynamic Query Builder** that would allow staff to find patients using real world identifiers (like name and address) rather than a hard to remember ID. To ensure **GDPR compliance**, the search requires multiple points of verification before sensitive medical data is revealed, ensuring that private information is only accessed after the patient's identity is confirmed. By using real world standards I will implement the **rule of two** before anyone can access data. For example the required inputs are: Firstname, Surname. Optional inputs can be added to refine searches and narrow down results.